# 02a Numerische Datentypen

## 02a Numerische Datentypen

00 Übersicht · Zurück: 02 Variablen · Weiter: 02b Zweierkomplement

### Ganzzahlen

| Datentyp | Bits | kleinster Wert | größter Wert | Konstanten |
|---|---|---|---|---|
| `int` | 32 | $-2^{31} =$ $-2.147.483.648$ | $2^{31} - 1 =$ $2.147.483.647$ | `Integer.MIN_VALUE` / `Integer.MAX_VALUE` |
| `long` | 64 | $-2^{63} =$ $-9.223.372.036.854.775.808$ | $2^{63} - 1 =$ $9.223.372.036.854.775.807$ | `Long.MIN_VALUE` / `Long.MAX_VALUE` |
| `short` | 16 | $-2^{15} = -32.768$ | $2^{15} - 1 = 32.767$ | `Short.MIN_VALUE` / `Short.MAX_VALUE` |
| `byte` | 8 | $-2^7 = -128$ | $2^7 - 1 = 127$ | `Byte.MIN_VALUE` / `Byte.MAX_VALUE` |

**Allgemein** bei $n$ Bit: $-2^{n-1}$ bis $2^{n-1} - 1$ (Grund: 02b Zweierkomplement).

> **Ein Zahlenwert** ohne Punkt** ist in Java automatisch ein `int`:**
> `int max = 100;`

### Gleitkommazahlen

| | `double` | `float` |
|---|---|---|
| Größe | 64 Bit (doppelte Genauigkeit) | 32 Bit (einfache Genauigkeit) |
| Wertebereich | $\pm$ 1,79769 $\cdot 10^{308}$ | $\pm$ 3,40282 $\cdot 10^{38}$ |
| Präzision | ca. **15** gültige Dezimalstellen | ca. **6 bis 7** gültige Dezimalstellen |
| Konstanten | `Double.MAX_VALUE`, `Double.MIN_VALUE` | `Float.MAX_VALUE`, `Float.MIN_VALUE` |
| Literal | Zahl **mit Punkt** ist automatisch `double`: `double price = 100.0;` | muss **explizit** mit `f`/`F` markiert werden: `float price = 100.0f;` |

### Schreibweise numerischer Literale

Gleitkommazahlen:

```
double value = 20.5;
double value = 0.0205E3;      // 0,0205 · 10^3
double value = 205000E-4;     // 205000 · 10^-4
```

Zuordnung des Datentyps:
| Literal | Typ |
|—|—|
| `20` | `int` |
| `20.0` | `double` |
| `20d` | `double` |
| `20f` | `float` |
| `20l` | `long` |
| `20b`, `20s` | **nicht möglich** |

### Rechenoperationen: Ergebnistyp

Bei gleichen Datentypen erhält man **nicht immer** den erwarteten Typ:

```
byte   + byte   = int
short  + short  = int
int    + int    = int
long   + long   = long
float  + float  = float
double + double = double
```

Bei **unterschiedlichen** Typen wird immer der **sichere (allgemeinere)** Typ als Ergebnis gewählt:

```
byte + short  = int
byte + long   = long
int  + float  = float
long + double = double
```

> **Empfehlung fürs erste Semester**
> `byte` und `short` sind für normale Berechnungen **ungeeignet** (gedacht für Umgebungen mit knappem Speicher). Verwende insbesondere **`int` und `double`**; `long` und `float` nur in Ausnahmefällen.

### Implizite Typkonversion

Findet immer statt, wenn
- ein Wert in einem bestimmten Typ **benötigt** wird,
- der Wert **nicht** in diesem Typ vorliegt **und**
- die Konversion **ohne Informationsverlust** möglich ist.

Um Informationsverlust zu beurteilen, betrachtet man die **Wertebereiche** der Typen.

Beispiele: `double price = 3;` · `int max = 3.0;` (99 Übungsaufgaben). Zu `char` `int`: 02c char und Zeichensätze.